

CSS 311 Parallel and Distributed Computing

Parallel Implementation of Medical Image Processing

Name 1: **Albin John**

Roll No: **2023BCD0005**

Name 2: **Rupsika Thipparthi**

Roll No: **2023BCS0197**

Course Instructor: **Dr. John Paul Martin**

Department of Computer Science **Indian Institute of Information
Technology Kottayam**

Date of Submission: [09/10/2025]

Project Abstract

The project Parallel implementation of medical image processing focuses on medical preprocessing tasks using Open MP parallelization. In traditional methods it is doing the preprocessing tasks like grayscale conversion, negative transformation and edge detection sequentially. This will lead to longer processing time for high resolution medical images.

This issue can be solved by parallelizing, we will distribute pixel wise computations across multiple threads. Each stage of processing like loading, transforming, and filtering is optimized to run concurrently without race conditions.

The Comparison between execution time of serial code and parallel code shows that the parallel code is 3 times faster for 6 image samples we took. If we increase the number of images and resolution of the image into a much higher value, the difference will be huge. This improvement enables faster preprocessing for diagnostic applications where timely image analysis is important.

1) Introduction

Medical image processing plays a major role in Modern Healthcare.

It's useful for Doctors , Radiologists for diagnosis purposes and for treating several diseases. There are many techniques such as MRIs , Xray, CT scanning

To get a useful information from these techniques like identifying tumors, tissue boundaries many image processing algorithms like Negative Transformation and Edge detection are used. Usually it takes a more execution time without Parallelising it because of its large size and high precision. The negative transformation is used to enhance visual contrast and highlight certain structures, whereas Sobel edge detection could highlight the boundaries and edges that could represent regions as tumors.

Parallel and distributed computing provide an efficient solution by dividing the work across multiple processor cores, thereby reducing execution time. This project speeds up image processing tasks, making it suitable for large scale diagnosis, especially in medical field time is a very important factor.

2. Literature Survey

1. Negative Transformation (Image Inversion)

Standard Serial Approach

The negative transformation is one of the simplest intensity transformation functions in digital image processing. The classical approach involves pixel-by-pixel operation with the formula: $s = (L-1) - r$, where s is output intensity, r is input intensity, and L is the maximum intensity level (256 for 8-bit images).

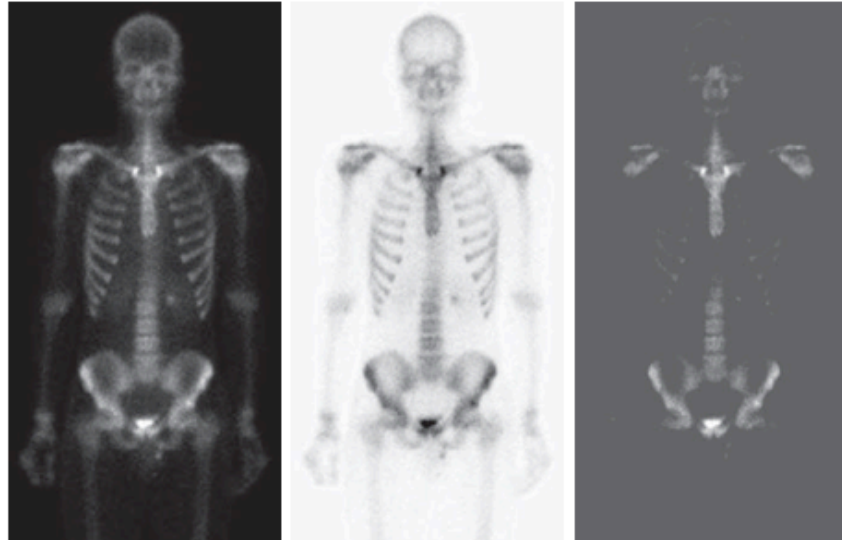
- **Gonzalez & Woods (2018)** present this as a fundamental point processing operation

$$A^c = \{(x, y, K - z) | (x, y, z) \in A\}$$

which is the set of pixels of A whose intensities have been subtracted from a constant K . This constant is equal to the maximum intensity value in the image, $2^k - 1$, where k is the number of bits used to represent z . Let A denote the 8-bit grayscale image in Fig. 2.36(a), and suppose that we want to form the negative of A using grayscale set operations. The negative is the set complement, and this is an 8-bit image, so all we have to do is let $K = 255$ in the set defined above:

$$A^c = \{(x, y, 255 - z) | (x, y, z) \in A\}$$

FIGURE 2.36
Set operations involving grayscale images. (a) Original image. (b) Image negative obtained using grayscale set complementation. (c) The union of image (a) and a constant image. (Original image courtesy of G.E. Medical Systems.)



The *union* of two grayscale sets A and B with the same number of elements is defined as the set

$$A \cup B = \left\{ \max_z(a, b) \mid a \in A, b \in B \right\}$$

**(Screenshots proofs from the book named Digital Image Processing FOURTH EDITION Rafael C. Gonzalez • Richard E. Woods)*

- Time complexity: $O(n \times m)$ where $n \times m$ are image dimensions
- **It is parallelizable due to pixel independence**

Limitations of Serial Implementation:

- Sequential processing becomes time-consuming for high-resolution images (e.g., 4K: 8.3 million pixels)
- Inefficient memory access patterns

2. Sobel Edge Detection

Standard Serial Approach

The Sobel filter consists of two 3×3 convolution kernels: one for detecting changes in the horizontal direction and the other in the vertical direction. These kernels compute the gradient of the image intensity at each pixel, highlighting the edges

2. THE SOBEL EDGE DETECTOR

The Sobel edge detection method is proposed by Sobel in 1970 [11]. The Sobel operator is the magnitude of the gradient computed by

$$M = \sqrt{s_x^2 + s_y^2}, \quad (1)$$

Where the partial derivative is computed by

$$s_x = (a_2 + ca_3 + a_4) - (a_0 + ca_7 + a_6) \quad (2)$$

$$s_y = (a_0 + ca_1 + a_2) - (a_6 + ca_5 + a_4) \quad (3)$$

With a constant $C=2$

Like the other gradient operators, s_x and s_y can be implemented using convolution masks:

$$s_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad s_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad (4)$$

The Sobel edge detector gives more emphasis on the image pixels that are closer to the center of the mask. This operator precedes the edges where the gradient is highest in grayscale images, this method also used in finding the

**(A screenshot from the paper publication by Neha Mathura, Shruti Mathur, Divya Mathur named A Novel Approach To Improve Sobel Edge Detector)*

Classical Algorithm:

1. Apply horizontal gradient mask (G_x)
2. Apply vertical gradient mask (G_y)
3. Compute gradient magnitude: $G = \sqrt{G_x^2 + G_y^2}$
4. Apply threshold for edge binary map

Time Complexity: $O(n \times m \times k^2)$ where k is kernel size (3×3 for Sobel)

3. Problem Statement and Objectives

3.1 Problem Definition

The Project take Raw Medical images as input and convert them into negative and Edge Detected images since Negative images enhance contrast by making light areas as dark and vice versa whereas Edge detection images highlight the boundaries and shapes of anatomical

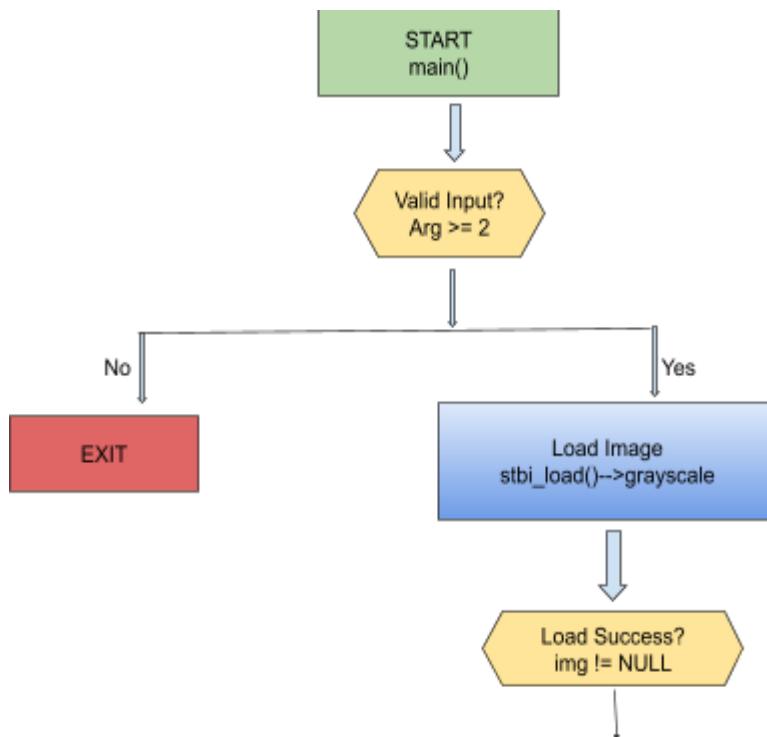
structures. All these operations are repetitive pixel computations so they can be parallelized by using Openmp for faster execution.

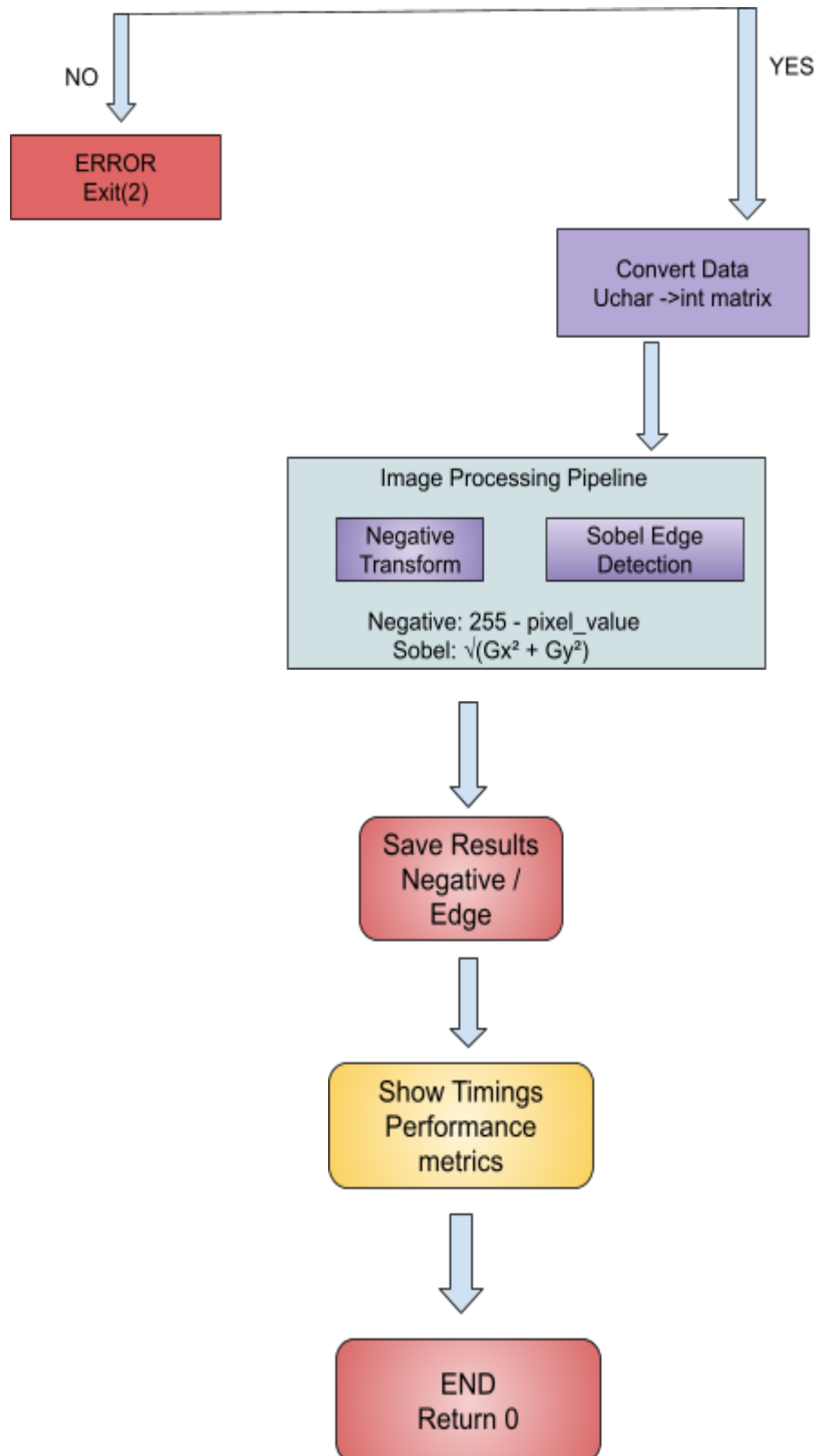
3.2 OBJECTIVES:

1. To collect medical images dataset for preprocessing.
2. To Implement Serial solution for creating Negative Transformation Images and Edge detection on medical images.
3. To Identify parallelizable code from serial.
4. To Implement Parallelized Solution for creating Negative Transformation Images and Edge detection on medical images.
5. To analyze and compare performance and scalability.

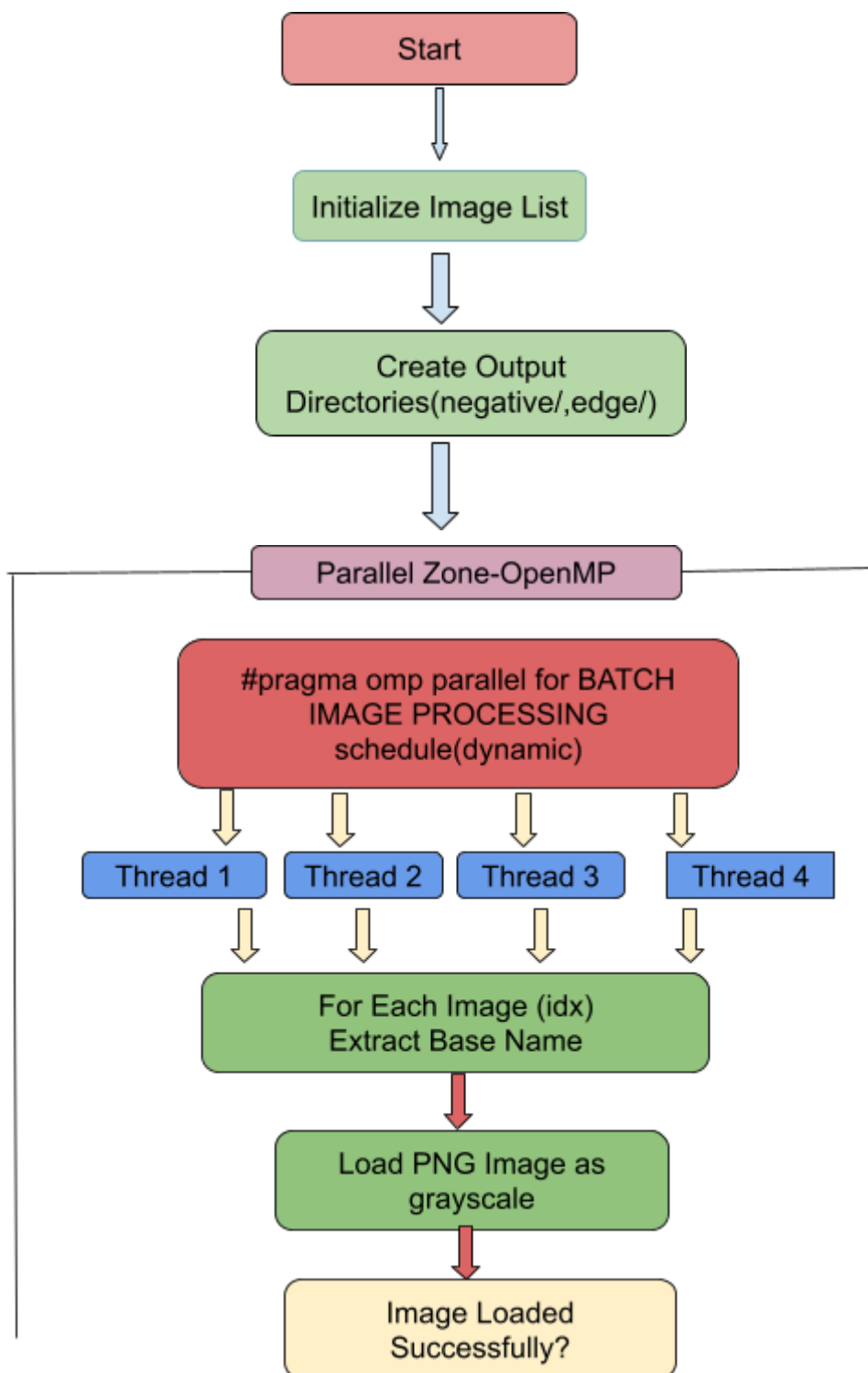
4. Methodology and System Architecture

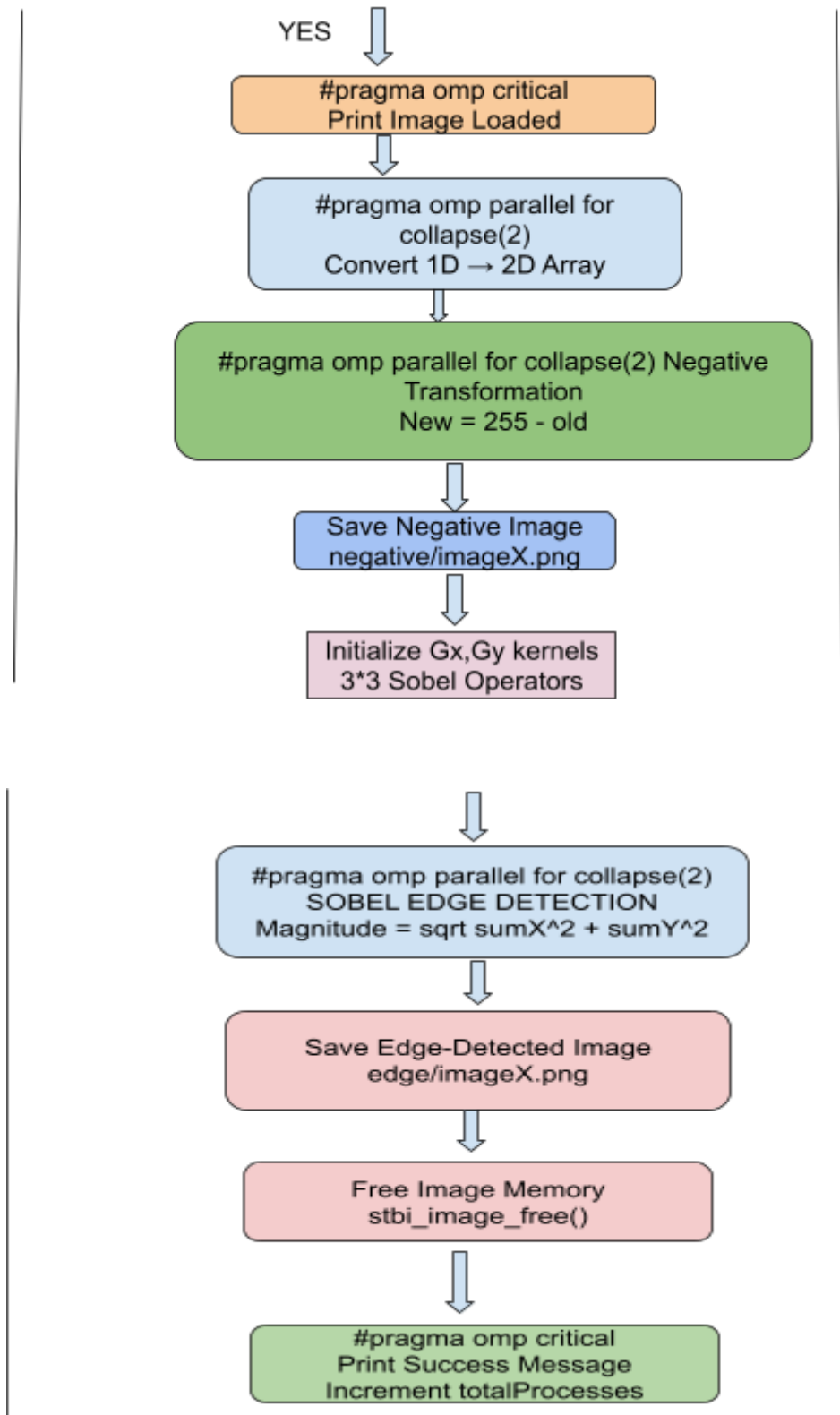
Flowchart For Serial Implementation

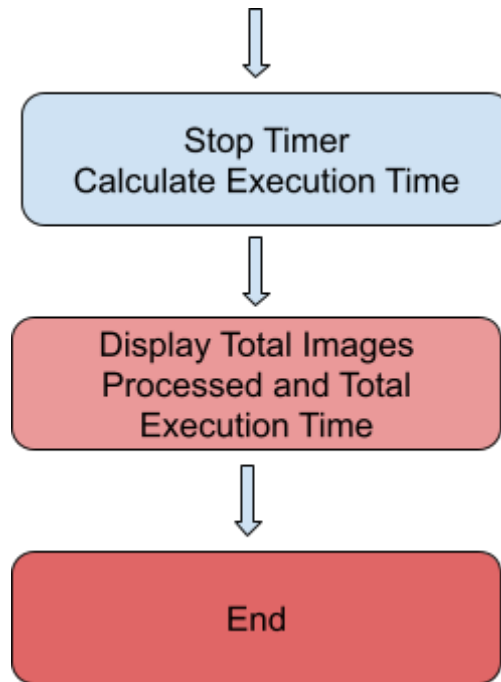




Flowchart For Parallel Implementation







4.1 Serial Algorithm :

Start timer

Initialize list of input image filenames

Create output folders: "negative" and "edge"

For each image file in the list:

Load image in grayscale mode

If image not loaded:

Print error and continue to next file

Convert 1D image array to 2D matrix

// Negative Transformation

For each pixel (i, j) in image:

negative_pixel = 255 - image[i][j]

Save the negative image to “negative” folder

// Sobel Edge Detection

Define Gx and Gy convolution kernels

For each pixel (i, j) excluding border:

sumX = sumY = 0

For m = -1 to 1:

For n = -1 to 1:

*sumX += image[i+m][j+n] * Gx[m+1][n+1]*

*sumY += image[i+m][j+n] * Gy[m+1][n+1]*

magnitude = sqrt(sumX² + sumY²)

If magnitude > 255:

magnitude = 255

edgesData[i][j] = magnitude

Save the edge image to “edge” folder

Free the memory of the current image

Increment total processed count

Stop timer

Print total number of images processed and total execution time

End

4.2 Parallel Algorithm (OpenMP):

Start timer

Initialize list of input image filenames

Create output folders: “negative” and “edge”

#pragma omp parallel for reduction(+:totalProcessed) schedule(dynamic)

For each image file in the list:

Load image in grayscale mode

If image not loaded:

Print error and continue

Convert 1D image array to 2D matrix

#pragma omp parallel for collapse(2)

For each pixel (i, j):

image[i][j] = grayscale value

// Parallel Negative Transformation

#pragma omp parallel for collapse(2)

For each pixel (i, j):

negative_pixel = 255 - image[i][j]

Save negative image to “negative” folder

// Parallel Sobel Edge Detection

Define Gx and Gy kernels

#pragma omp parallel for collapse(2)

For each pixel (i, j) excluding border:

sumX = sumY = 0

For m = -1 to 1:

For n = -1 to 1:

*sumX += image[i+m][j+n] * Gx[m+1][n+1]*

*sumY += image[i+m][j+n] * Gy[m+1][n+1]*

magnitude = sqrt(sumX² + sumY²)

If magnitude > 255:

magnitude = 255

edgesData[i][j] = magnitude

Save edge image to “edge” folder

Free memory of current image

Increment total processed count

Stop timer

Print total number of images processed and total execution time

End

4.3 Implementation Details

Programming Language: **C/C++**

Platform: **Windows**

Compiler: **GCC (-fopenmp)**

Hardware Configuration: **CPU: Intel Core i5 / i7 or equivalent**

Cores/Threads: 4–8 (depending on the system)

RAM: 8 GB or higher

Disk: SSD recommended for faster I/O operations

5. Results and Analysis

Serial Output:

```
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> g++ -fopenmp medical_serial.cpp -o medical_serial
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> ./medical_serial
Albin John - 2023BCD0005
Rupsika T - 2023BCS0197
Image loaded: image1.png - 512x512 (channels=1)
Processed image1.png: Saved negative/image1.png and edge/image1.png
Image loaded: image2.png - 512x512 (channels=1)
Processed image2.png: Saved negative/image2.png and edge/image2.png
Image loaded: image3.png - 512x512 (channels=1)
Processed image3.png: Saved negative/image3.png and edge/image3.png
Image loaded: image4.png - 512x512 (channels=1)
Processed image4.png: Saved negative/image4.png and edge/image4.png
Image loaded: image5.png - 512x512 (channels=1)
Processed image5.png: Saved negative/image5.png and edge/image5.png
Image loaded: image6.png - 512x512 (channels=1)
Processed image6.png: Saved negative/image6.png and edge/image6.png
Batch processing complete! 6 images processed.
Total completion time: 0.321 seconds
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project>
```

Parallel Output:

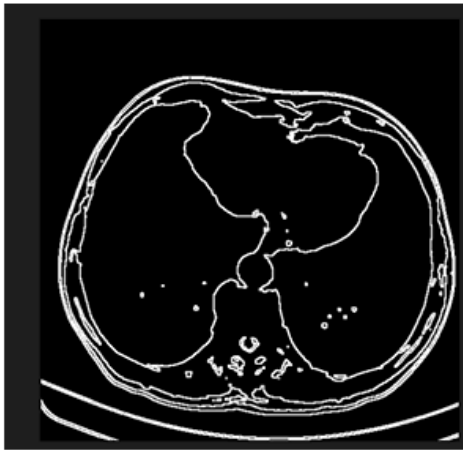
```
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> g++ -fopenmp medical_parallel.cpp -o medical_parallel
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> ./medical_parallel
Albin John - 2023BCD0005
Rupsika T - 2023BCS0197
Image loaded: image1.png - 512x512
Image loaded: image6.png - 512x512
Image loaded: image2.png - 512x512
Image loaded: image3.png - 512x512
Image loaded: image5.png - 512x512
Image loaded: image4.png - 512x512
Processed image1.png: Saved negative/image1.png and edge/image1.png
Processed image6.png: Saved negative/image6.png and edge/image6.png
Processed image2.png: Saved negative/image2.png and edge/image2.png
Processed image4.png: Saved negative/image4.png and edge/image4.png
Processed image3.png: Saved negative/image3.png and edge/image3.png
Processed image5.png: Saved negative/image5.png and edge/image5.png
Batch processing complete! 6 images processed.
Total completion time: 0.098 seconds
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project>
```

Test Case 1:

Input:



Edge image



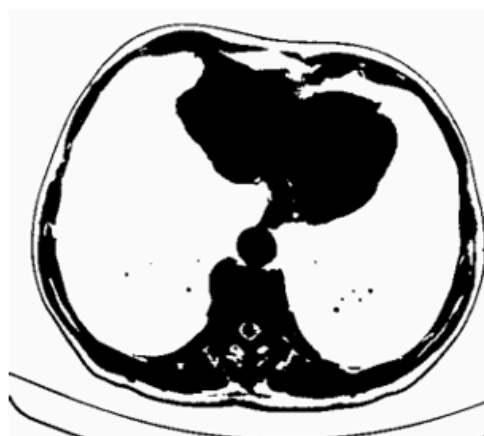
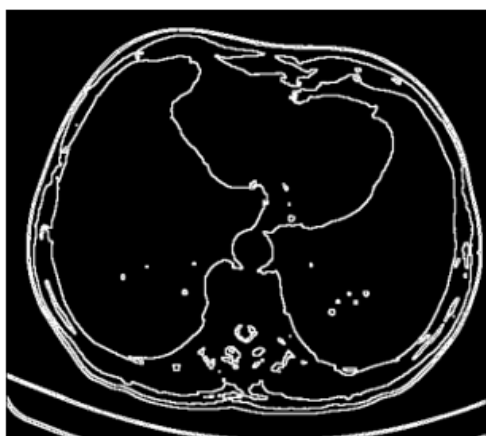
Negative Image



Test Case 2:



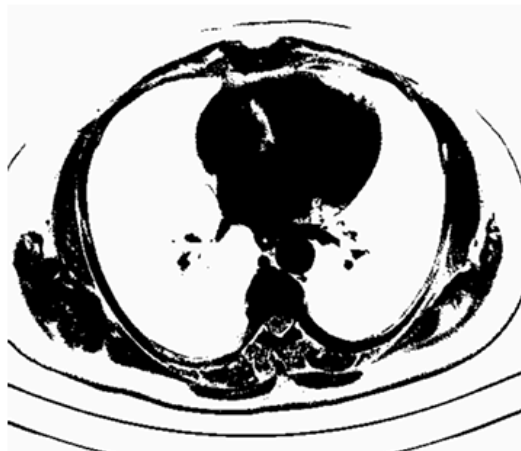
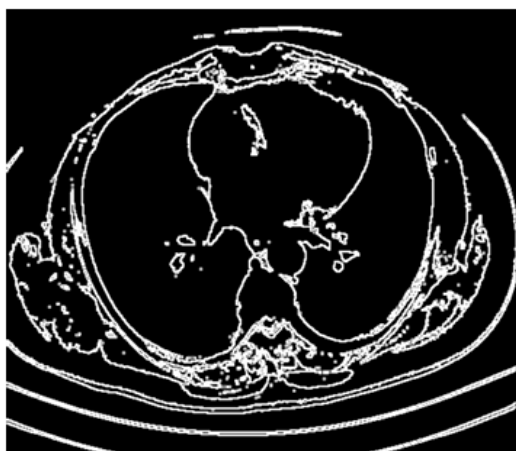
Output Images



Test Case 3:



Output Images



Performance Comparison Table:

Implementation	Execution Time (seconds)	SpeedupT
Serial	0.321	1×
Parallel (OpenMP)	0.098	3.28×

$$\text{Speedup} = \text{Serial Time} / \text{Parallel Time} = 0.321 / 0.098 \approx 3.28$$

Time Complexity Analysis:

Negative Transformation:

Each pixel is processed independently $\rightarrow O(H \times W)$

H = image height, W = image width

Sobel Edge Detection:

Each pixel (excluding border) requires a 3×3 convolution $\rightarrow O(H \times W \times 9) \approx O(H \times W)$

Batch Processing (N images):

Total complexity $\approx O(N \times H \times W)$

Scalability

Execution time is reduced by parallelizing code with OpenMp

Speedup is mainly depends on the number of CPU cores and memory.

Usually images of large size,for more number of images have very good Scalability because pixel wise operations are highly parallelizable

Some overhead occurs due to thread creation and I/O operations, but it is negligible for larger number of images and for larger batches.

6. Discussion and Observations

This implementation shows a significant improvement in performance when using OpenMP parallelization. For a batch of six images, the serial execution time was 0.321 seconds, while in the parallel version completed in 0.098 seconds, acquiring a speedup of 3.28×.

This shows that independent pixel-wise operations, such as negative transformation and Sobel edge detection, are highly efficient when parallelization.

Limitations

Here we are parallelizing the number of images processed simultaneously, But we are not parallelizing the edge detection and negative transformation per image,

For example, Image 1, Image 2, Image 3 all are executing parallelly. But if we go inside image 1 after completing negative transformation only it is doing edge detection, these both transformations are not doing simultaneously even though these both are not dependent . If we can parallelize it we can improve the efficiency much higher.

But the problem we are facing is when we try to parallelize it, the whole code is executing in thread 0, Because the nested parallelization is not supported in openMP. The execution time is getting larger when we tried to solve that issue using AI.

7. Conclusion and Future Scope

In this project, a medical image processing pipeline was implemented to carry out grayscale conversion, negative transformation, and Sobel edge detection on batches of PNG images. A serial implementation was initially done followed by an OpenMP-based parallel version with a

speedup of 3.28× for six sample images. The parallel implementation performs effective pixel-wise distribution across several CPU cores with superior execution time and scalability for larger data.

Future Scope:Appendix A. Serial Code and Output B. Parallel Code (OpenMP) and Output

The project can be extended to MPI in a way that large-scale medical image analysis is possible on multiple machines for a broader area.

CUDA acceleration of GPU could also decrease computation time specifically for high resolution images and real time use.

Other preprocessing methods such as noise removal, histogram equalization and adaptive thresholding may be incorporated to enhance medical diagnostic precision.

This project serves as a foundation for scalable and effective medical image preprocessing pipelines that are applicable in research and clinical pipelines.

8. References:

- *Digital Image Processing FOURTH EDITION Rafael C. Gonzalez • Richard E. Woods -*

[Digital Image Processing. 4e](#)

- *Paper publication by Neha Mathura, Shruti Mathur ,Divya Mathur named A Novel Approach To Improve Sobel Edge Detector*

[A Novel Approach to Improve Sobel Edge Detector - ScienceDirect](#)

- *A Descriptive Algorithm for Sobel Image Edge Detection, O. R. Vincent, O. Folorunso*

<https://proceedings.informingscience.org/InSITE2009/InSITE09p097-107Vincent613.pdf>

Appendix

A. Serial Code

```
#define STB_IMAGE_IMPLEMENTATION

#define STB_IMAGE_WRITE_IMPLEMENTATION

#include "stb_image.h"

#include "stb_image_write.h"


#include <iostream>

#include <vector>

#include <cmath>

#include <filesystem>

#include <string>

#include <cstring>

#include <omp.h>

using namespace std;


int main() {

    cout << "Albin John - 2023BCD0005\n";

    cout << "Rupsika T - 2023BCS0197\n";

    double start_time = omp_get_wtime();


    // List of input image filenames

    vector<string> inputFilenames = {"image1.png", "image2.png",
    "image3.png", "image4.png", "image5.png", "image6.png"}; // Add more as needed


    // Create output folders if they don't exist
```

```

std::filesystem::create_directory("negative");

std::filesystem::create_directory("edge");


int totalProcessed = 0;

for (const auto& inputFilename : inputFileNames) {

    int width, height, channels;


    // Extract base filename (without path)

    string inputPath(inputFilename);

    size_t pos = inputPath.find_last_of("\\");

    string baseName = (pos == string::npos) ? inputPath : inputPath.substr(pos + 1);


    // Load PNG (converted to grayscale)

    unsigned char* img = stbi_load(inputFilename.c_str(), &width, &height, &channels, 1);

    if (!img) {

        cerr << "Error: Could not load " << inputFilename << "\n";

        continue;

    }


    cout << "Image loaded: " << inputFilename << " - " << width << "x" << height << "
(channels=" << channels << ")\\n";


    // Convert to 2D vector

    vector<vector<int>> image(height, vector<int>(width));

    for (int i = 0; i < height; i++) {

        for (int j = 0; j < width; j++) {

            image[i][j] = img[i * width + j]; // grayscale value (0–255)

```

```

    }
}

// Negative transformation

vector<unsigned char> negativeData(width * height);

for (int i = 0; i < height; i++) {
    for (int j = 0; j < width; j++) {
        negativeData[i * width + j] = 255 - image[i][j];
    }
}

string negativePath = "negative/" + baseName;

stbi_write_png(negativePath.c_str(), width, height, 1, negativeData.data(), width);


// Sobel edge detection

int Gx[3][3] = { {-1,0,1}, {-2,0,2}, {-1,0,1} };
int Gy[3][3] = { {-1,-2,-1}, {0,0,0}, {1,2,1} };

vector<unsigned char> edgesData(width * height, 0);

for (int i = 1; i < height - 1; i++) {
    for (int j = 1; j < width - 1; j++) {
        int sumX = 0, sumY = 0;

        for (int m = -1; m <= 1; m++) {
            for (int n = -1; n <= 1; n++) {
                int pixel = image[i+m][j+n];

                sumX += pixel * Gx[m+1][n+1];
                sumY += pixel * Gy[m+1][n+1];
            }
        }
    }
}

```

```

        }

    }

    int mag = sqrt(sumX*sumX + sumY*sumY);

    if (mag > 255) mag = 255;

    edgesData[i * width + j] = mag;

}

}

string edgePath = "edge/" + baseName;

stbi_write_png(edgePath.c_str(), width, height, 1, edgesData.data(), width);


    cout << "Processed " << inputFilename << ": Saved " << negativePath << " and " <<
edgePath << "\n";

    stbi_image_free(img);

    totalProcessed++;

}

double end_time = omp_get_wtime();

cout << "Batch processing complete! " << totalProcessed << " images processed.\n";

cout << "Total completion time: " << (end_time - start_time) << " seconds\n";

return 0;

}

```

Output

```

PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> g++ -fopenmp medical_serial.cpp -o medical_serial
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> ./medical_serial
Albin John - 2023BCD0005
Rupsika T - 2023BCS0197
Image loaded: image1.png - 512x512 (channels=1)
Processed image1.png: Saved negative/image1.png and edge/image1.png
Image loaded: image2.png - 512x512 (channels=1)
Processed image2.png: Saved negative/image2.png and edge/image2.png
Image loaded: image3.png - 512x512 (channels=1)
Processed image3.png: Saved negative/image3.png and edge/image3.png
Image loaded: image4.png - 512x512 (channels=1)
Processed image4.png: Saved negative/image4.png and edge/image4.png
Image loaded: image5.png - 512x512 (channels=1)
Processed image5.png: Saved negative/image5.png and edge/image5.png
Image loaded: image6.png - 512x512 (channels=1)
Processed image6.png: Saved negative/image6.png and edge/image6.png
Batch processing complete! 6 images processed.
Total completion time: 0.321 seconds
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project>

```

B. Parallel Code (OpenMP)

```

#define STB_IMAGE_IMPLEMENTATION

#define STB_IMAGE_WRITE_IMPLEMENTATION

#include "stb_image.h"

#include "stb_image_write.h"

#include <iostream>

#include <vector>

#include <cmath>

#include <filesystem>

#include <string>

#include <omp.h>

using namespace std;

int main() {

    double start_time = omp_get_wtime();

    cout << "Albin John - 2023BCD0005\n";

    cout << "Rupsika T - 2023BCS0197\n";

```

```

// List of input images

vector<string> inputFileNames = {"image1.png", "image2.png", "image3.png",
"image4.png", "image5.png", "image6.png"};


// Create output folders

std::filesystem::create_directory("negative");

std::filesystem::create_directory("edge");


int totalProcessed = 0;


#pragma omp parallel for reduction(+:totalProcessed) schedule(dynamic)
for (int idx = 0; idx < inputFileNames.size(); idx++) {

    const string& inputFilename = inputFileNames[idx];

    int width, height, channels;


    // Extract base name

    size_t pos = inputFilename.find_last_of("\\");

    string baseName = (pos == string::npos) ? inputFilename : inputFilename.substr(pos +
1);


    // Load PNG as grayscale

    unsigned char* img = stbi_load(inputFilename.c_str(), &width, &height, &channels, 1);

    if (!img) {

        #pragma omp critical

        cerr << "Error: Could not load " << inputFilename << "\n";

        continue;

```



```
}
```

```
#pragma omp critical
```

```
cout << "Image loaded: " << inputFilename << " - " << width << "x" << height << "\n";
```

```
// Convert to 2D vector
```

```
vector<vector<int>> image(height, vector<int>(width));
```

```
#pragma omp parallel for collapse(2)
```

```
for (int i = 0; i < height; i++) {
```

```
    for (int j = 0; j < width; j++) {
```

```
        image[i][j] = img[i * width + j];
```

```
    }
```

```
}
```

```
//
```

```
vector<unsigned char> negativeData(width * height);
```

```
#pragma omp parallel for collapse(2)
```

```
for (int i = 0; i < height; i++) {
```

```
    for (int j = 0; j < width; j++) {
```

```
        negativeData[i * width + j] = 255 - image[i][j];
```

```
    }
```

```
}
```

```
string negativePath = "negative/" + baseName;
```

```
stbi_write_png(negativePath.c_str(), width, height, 1, negativeData.data(), width);
```

```

int Gx[3][3] = { {-1,0,1}, {-2,0,2}, {-1,0,1} };
int Gy[3][3] = { {-1,-2,-1}, {0,0,0}, {1,2,1} };

vector<unsigned char> edgesData(width * height, 0);

#pragma omp parallel for collapse(2)
for (int i = 1; i < height - 1; i++) {
    for (int j = 1; j < width - 1; j++) {
        int sumX = 0, sumY = 0;
        for (int m = -1; m <= 1; m++) {
            for (int n = -1; n <= 1; n++) {
                int pixel = image[i+m][j+n];
                sumX += pixel * Gx[m+1][n+1];
                sumY += pixel * Gy[m+1][n+1];
            }
        }
        int mag = sqrt(sumX*sumX + sumY*sumY);
        if (mag > 255) mag = 255;
        edgesData[i * width + j] = mag;
    }
}

string edgePath = "edge/" + baseName;
stbi_write_png(edgePath.c_str(), width, height, 1, edgesData.data(), width);

stbi_image_free(img);

#pragma omp critical

```

```

    {

        cout << "Processed " << inputFilename << ": Saved " << negativePath << " and " <<
edgePath << "\n";

        totalProcessed++;

    }

}

double end_time = omp_get_wtime();

cout << "Batch processing complete! " << totalProcessed << " images processed.\n";

cout << "Total completion time: " << (end_time - start_time) << " seconds\n";

return 0;

}

```

Output

```

PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> g++ -fopenmp medical_parallel.cpp -o medical_parallel
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project> ./medical_parallel
Albin John - 2023BCD0005
Rupsika T - 2023BCS0197
Image loaded: image1.png - 512x512
Image loaded: image6.png - 512x512
Image loaded: image2.png - 512x512
Image loaded: image3.png - 512x512
Image loaded: image5.png - 512x512
Image loaded: image4.png - 512x512
Processed image1.png: Saved negative/image1.png and edge/image1.png
Processed image6.png: Saved negative/image6.png and edge/image6.png
Processed image2.png: Saved negative/image2.png and edge/image2.png
Processed image4.png: Saved negative/image4.png and edge/image4.png
Processed image3.png: Saved negative/image3.png and edge/image3.png
Processed image5.png: Saved negative/image5.png and edge/image5.png
Batch processing complete! 6 images processed.
Total completion time: 0.098 seconds
PS C:\Users\ALBIN JOHN\OneDrive\Desktop\Parallel programming\Project>

```

Thank You !!